REPORTING DES RÉSULTATS DU PoC

Validation du service d’orientation hospitalière
en situation d’urgence

Ce rapport expose les résultats fonctionnels, la validation du logiciel, les mesures de charge, la couverture des objectifs et les risques restant à maîtriser. Les conclusions sont limitées au périmètre technique effectivement vérifié.

1 290	7	100 000
établissements simulés	tests backend réussis	requêtes de charge

DÉCISION TECHNIQUE

Faisabilité démontrée sur un périmètre local et simulé. La poursuite du projet est justifiée ; la qualification clinique, la sécurité de production et la résilience du routage exigent des travaux complémentaires.

Dépôt du code : https://github.com/naegel23/P11-MedHead-PoC

Dépôt d’architecture : https://github.com/naegel23/P11-MedHead-Architecture

01 / SYNTHÈSE

Synthèse exécutive

La preuve de concept vérifie une capacité d'orientation hospitalière ; elle ne remplace pas une qualification métier ou clinique.

Fonctionnel	Établi	Recherche, filtres et affichage Vue.js vérifiés localement.
Qualité logicielle	Établi	Tests automatisés backend et frontend ; CI opérationnelle.
Charge backend	Établi	998,93 req/s ; 0 % d'erreur ; maximum observé 137 ms.
Routage externe	Partiel	Recommandation dépendante d'OpenRouteService et de ses quotas.
Continuité / sécurité	Partiel	Exécution Docker et isolation ; exigences de production non couvertes.
KPI métier	Non mesuré	Taux d'orientation et délai de prise en charge non mesurés.

Constat principal

Le backend assure les fonctions de recherche sur une base H2 simulée et reste performant dans le scénario local soumis à JMeter. Le parcours via le navigateur et l'interface Vue.js a également été exercé.

Décision de poursuite

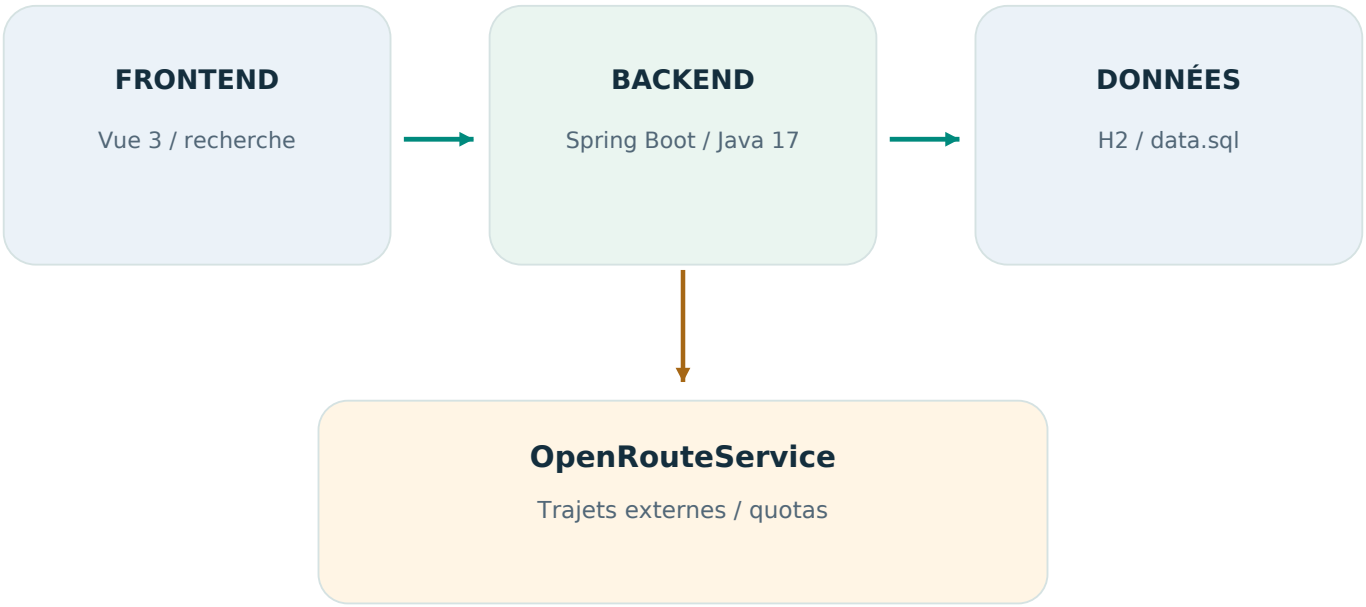
Poursuivre l'industrialisation sous conditions : garantir les dépendances externes, sécuriser les flux, définir une architecture résiliente, automatiser les parcours critiques et mesurer les résultats métier.

02 / PÉRIMÈTRE

Périmètre et architecture évaluée

La solution combine un frontend web, une API REST Java, des données hospitalières simulées et un fournisseur de calcul de trajets.

CHAÎNE FONCTIONNELLE DU POC



COMPOSANTS ET PREUVE DE FONCTIONNEMENT

API REST	GET /hospital ; filtrage par spécialité, lits et rayon.
Documentation	Contrat OpenAPI sur /v3/api-docs ; Swagger UI accessible.
Interface web	Saisie des critères, consommation du backend et affichage des résultats.
Conteneurisation	Image Docker backend exécutée avec réponse HTTP 200.
Qualité & intégration	Tests Maven et Vitest ; exécution des pipelines GitHub Actions.

Frontière du PoC. Un service Spring Boot principal et une interface web ; pas de réservation réelle de lit, de bus de messages, de plateforme de microservices complète ou de traitement de dossiers patients.

03 / FONCTIONNALITÉS

Résultats de la validation fonctionnelle

Les résultats portent sur des réponses calculées à partir du jeu de test et des paramètres transmis par l'utilisateur.

CAPACITÉ	ÉTAT	CONSTATS
Recherche multi-critères	Vérifié	Localisation, spécialité et rayon saisis et transmis.
Liste des établissements	Vérifié	1 290 lignes chargées en configuration de référence.
Disponibilité simulée	Vérifié	Filtrage sur l'attribut freebed et la spécialité.
Rayon géographique	Vérifié	Filtrage spatial sur l'endpoint local dédié.
Interface Vue.js	Vérifié	Recherche locale ; HTTP 200 ; 20 résultats affichés.
Itinéraire /nearest	Partiel	Service externe requis ; cas de limitation observés.
Réservation du lit	Hors PoC	Aucune réservation réelle ni événement publié.

Parcours interface → API observé

Recherche sur le périmètre pharmacology, avec longitude -2.0115245, latitude 52.00514984 et rayon de 100 km. L'interface locale a obtenu HTTP 200 et affiché 20 établissements.

Interprétation : le parcours prouve l'interconnexion du frontend et du backend en local. Les tests unitaires d'interface utilisent également des réponses simulées. Une campagne E2E automatisée contre tous les composants réels reste à construire.

04 / DONNÉES

Jeu d’essai et reproductibilité

Les données servent exclusivement à démontrer la recherche et le filtrage. Elles ne décrivent pas l’occupation réelle des hôpitaux.

1 290	0–150	H2
établissements chargés	lits simulés par entrée	persistance locale

PROVENANCE ET TRANSFORMATIONS

La source CSV indiquée pour la préparation des établissements est le répertoire hospitalier du NHS : https://assets.nhs.uk/data/foi/Hospital.csv. Cette provenance est renseignée dans la documentation du jeu de test ; la campagne de validation ne comporte pas d’audit indépendant du CSV source.

- Les noms, adresses et coordonnées géographiques servent à identifier et localiser les établissements de référence.
- Les spécialités sont attribuées de manière simulée à partir d’une nomenclature de spécialités NHS.
- Le champ **freebed** est une valeur générée entre 0 et 150 et ne constitue pas une disponibilité réelle.
- Les enregistrements sont définis dans **data.sql** et chargés dans la base H2 au démarrage de l’application.

LECTURE DES CHAMPS

hospital_name / address	Identification et présentation du site hospitalier.
longitude / latitude	Périmètre spatial et point de départ du routage.
spec	Filtrage des compétences médicales simulées.
freebed	Sélection des entrées avec lits simulés disponibles.

Conséquence sur les conclusions

Ce jeu permet de reproduire les comportements logiciels, mais ne peut justifier une décision médicale, confirmer une disponibilité hospitalière ou calculer un taux de bonne orientation en situation réelle.

05 / VALIDATION LOGICIELLE

Tests automatisés et intégration continue

L’intégration continue fournit une validation reproductible des composantes logicielles sur le périmètre couvert par les suites de tests.

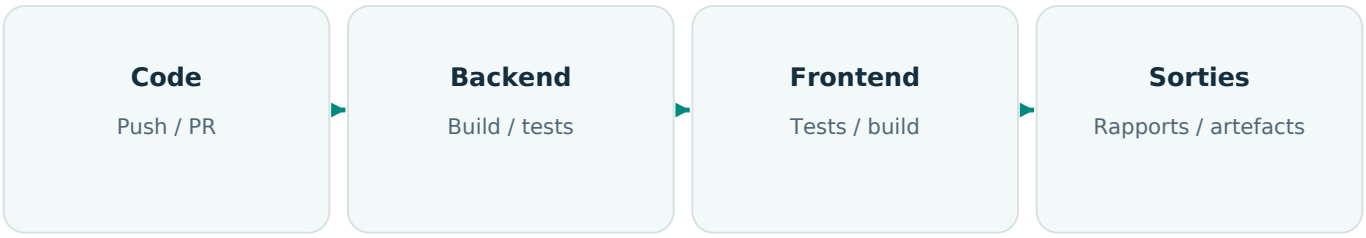
Backend / Maven

7 tests automatisés réussis. Les suites sont exécutées par Maven ; la commande de vérification complète du backend a terminé sans erreur.

Frontend / Vitest

Tests Vue.js exécutés avec Vitest et Vue Test Utils. Les scénarios incluent l’affichage du formulaire et le comportement de recherche sur réponse simulée.

CHAÎNE D’INTÉGRATION



- GitHub Actions déclenché par les **push** et **pull requests** sur la branche main.
- Exécution du backend avec **Java 17** ; compilation et contrôles Maven.
- Exécution du frontend avec **Node.js 22** ; tests Vitest et compilation Vite.
- Exécution sur runner Ubuntu 24.04 ; journaux et artefacts consultables dans GitHub Actions.

Autres validations observées

OpenAPI /v3/api-docs : HTTP 200. Swagger UI /swagger-ui/index.html : HTTP 200. Backend lancé dans une image Docker Java 17 : requête API retournant HTTP 200.

Limite : la disponibilité du workflow et la construction des artefacts établissent une intégration continue fonctionnelle ; elles ne démontrent pas un déploiement continu dans une infrastructure de production.

06 / PERFORMANCE

Scénario et protocole de charge

Une campagne Apache JMeter sollicite le backend local sur une recherche filtrée, sans dépendance au routage externe.

ENDPOINT MESURÉ

GET /hospital/free/speciality/pharmacology/range/-2.0115245/52.00514984/100

Filtres : spécialité pharmacology • lits simulés disponibles • rayon 100 km

100	1 000	100 000
utilisateurs virtuels	boucles par utilisateur	requêtes au total

CRITÈRE DE RÉFÉRENCE

La cible est une capacité de traitement d'au moins **800 requêtes par seconde** avec un **temps de réponse inférieur à 200 ms** sur un service. Le plan JMeter a été configuré pour viser ce débit ; le débit effectivement observé est reporté séparément.

Cadre expérimental

Campagne locale, sur un backend alimenté par la base H2 de test. Aucun appel à OpenRouteService n'est inclus dans les mesures. L'infrastructure employée n'est pas considérée comme représentative de la production.

LIMITES DE REPRODUCTIBILITÉ

- Conserver le plan JMeter, le jeu **data.sql**, le profil d'exécution et les résultats du run pour répéter la campagne.
- Comparer le débit mesuré et les percentiles, plutôt que de conclure à partir d'une valeur moyenne uniquement.
- Retester séparément le scénario intégrant le routage externe, avec une politique de quotas et de repli définie.

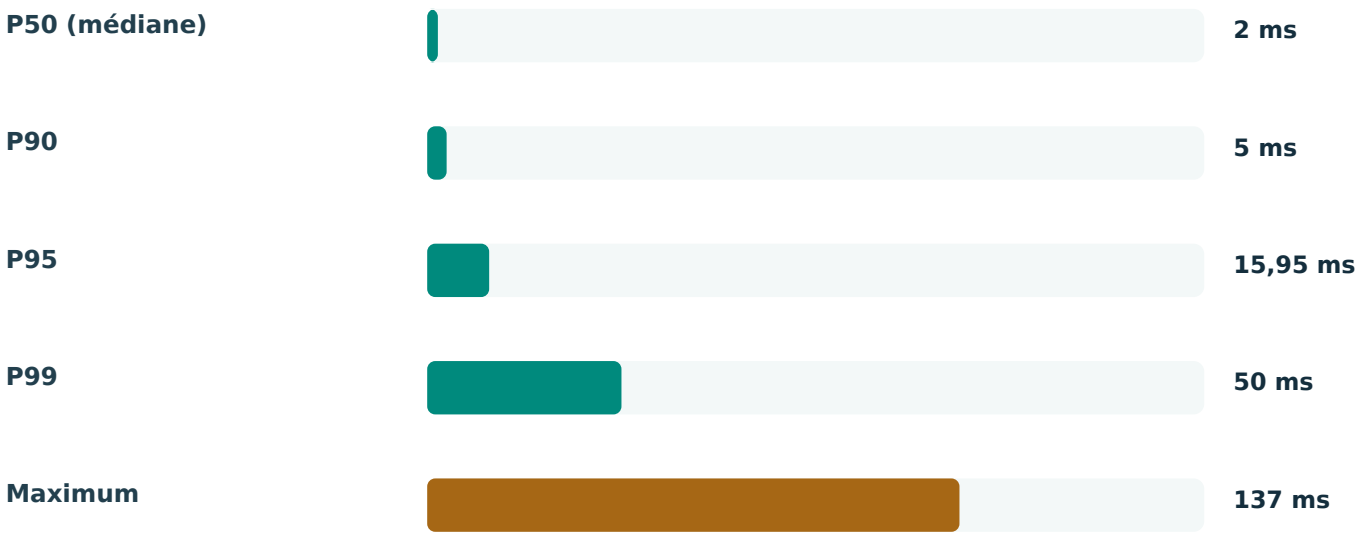
07 / PERFORMANCE

Résultats JMeter

Mesures du run local de 100 000 requêtes sur le seul endpoint de recherche filtrée.

998,93	0 %	3,98 ms
requêtes / seconde	erreurs HTTP observées	latence moyenne

DISTRIBUTION DES TEMPS DE RÉPONSE



Échelle des barres : 0 à 200 ms. Le seuil de référence est de 200 ms ; toutes les valeurs relevées pour ce run lui sont inférieures.

CRITÈRE LOCAL

Validé dans les conditions du test. Le débit de 998,93 req/s dépasse la référence de 800 req/s ; 100 000 requêtes ont été traitées sans erreur et le maximum mesuré est de 137 ms.

Portée : ce résultat n'établit ni une capacité garantie en production, ni la performance de la route **/nearest**, ni la disponibilité en présence d'un fournisseur externe limité.

08 / OBJECTIFS

Évaluation des objectifs de l’hypothèse

Les indicateurs métier, les propriétés techniques et les capacités de livraison doivent être évalués séparément.

OBJ-01 — **Orientation correcte supérieure à 90 %** — Non mesuré

Les données simulées ne fournissent ni cas cliniques réels ni référentiel de bonnes orientations.

OBJ-02 — **Délai moyen 18,25 → 12 minutes** — Non mesuré

Aucun parcours opérationnel complet n’a été chronométré avant et après déploiement.

OBJ-03 — **800 requêtes/s et moins de 200 ms** — Démontré localement

998,93 req/s ; 0 % d’erreur ; maximum 137 ms sur l’endpoint local hors routage.

OBJ-04 — **API, documentation et tests** — Établi

REST/OpenAPI, interface Vue.js, tests automatisés et exécution Docker disponibles.

OBJ-05 — **Réutilisation et intégration continue** — Partiel

GitHub Actions fonctionne ; déploiement production et événements métier à intégrer.

Conclusion sur les KPI

La validation technique contribue à réduire le risque d’implémentation. Elle ne permet pas de conclure à un gain clinique, à l’atteinte du taux de bonnes orientations ou au délai moyen cible.

09 / RISQUES

Risques et limites identifiés

Les écarts ci-dessous doivent être levés avant d'envisager l'usage opérationnel du service dans un contexte hospitalier.

R-01 Dépendance au routage — Élevé

Constat. OpenRouteService a répondu 429 sous sollicitation ; certains appels /nearest peuvent aboutir à 408.

Traitement. Définir quotas, backoff borné, cache, disjoncteur et mode dégradé.

R-02 Données non opérationnelles — Élevé

Constat. Les spécialités et lits sont simulés ; aucune synchronisation avec des hôpitaux réels.

Traitement. Définir flux d'alimentation fiables, fraîcheur et qualité des données.

R-03 Protection des accès — Élevé

Constat. Authentification, autorisation et sécurité transport de production non démontrées.

Traitement. Déployer IAM, HTTPS, secrets gérés et contrôles de sécurité.

R-04 Tolérance aux pannes — Élevé

Constat. Pas de redondance active, basculement ni plan de reprise testé.

Traitement. Construire une plateforme résiliente et exécuter des tests de rupture.

R-05 Interopérabilité événementielle — Moyen

Constat. Aucun événement métier de réservation et aucun bus intégré.

Traitement. Définir contrats d'événements, livraison fiable et idempotence.

Vigilance : les réponses HTTP du fournisseur externe révèlent une contrainte architecturale structurante. Le débit mesuré sur les appels internes ne peut pas être appliqué mécaniquement au parcours /nearest.

10 / DÉCISION ET SUITES

Décision et plan de consolidation

Les éléments démontrés justifient une poursuite encadrée, sans assimiler le démonstrateur à un système médical exploitable.

TRAVAUX PRIORITAIRES

P1	**Sécuriser la chaîne de routage**	Quotas, retry avec backoff, cache, dégradation contrôlée et supervision.
P1	**Fiabiliser les données**	Contrats de synchronisation, disponibilité réelle et traçabilité des mises à jour.
P1	**Renforcer la sécurité**	IAM, contrôle d'accès, HTTPS, secrets, audit et évaluation réglementaire.
P2	**Valider la résilience**	Infrastructure représentative, redondance, reprise et scénarios de panne.
P2	**Compléter l'automatisation**	Tests E2E réels, tests de contrat, tests de non-régression et charge externe.
P2	**Consolider l'intégration**	Événements métier, observabilité et interfaces avec les autres domaines.
P3	**Mesurer l'impact métier**	Campagne contrôlée sur orientations correctes et délais opérationnels.

Décision proposée

PoC techniquement concluant pour la recherche filtrée et son intégration logicielle. Autoriser l'étape de consolidation architecturale et de qualification, sans mise en production clinique à ce stade.

Critères de sortie : routage tolérant aux erreurs, données hospitalières qualifiées, sécurité démontrée, disponibilité et reprise testées, parcours critiques automatisés et KPI métier évalués sur un protocole représentatif.